Quiz — 2.1.3 Thinking Procedurally — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Answer key

Section A (6 marks — 1 each) 1. **B** — splitting one big problem into smaller sub-problems is decomposition. 2. **B** — water must be boiled before it can be poured; ordering with a dependency. 3. **B** — a named, self-contained, callable block of code is a sub-procedure (subroutine). 4. **B** — picking up depends on having navigated there, so the steps are sequential/fixed order. 5. **B** — decomposition lets sub-problems be shared out among different developers. 6. **B** — the tax step consumes the gross pay output by the previous step, so it must follow it.

Section B (9 marks)

7. **[3]** Any three (1 each): accept/validate coins or card payment; let user select a product; check stock/availability; dispense the item; give change; update stock count; display messages. Must be plausible sub-tasks of the vending scenario.

8. **[2]** Any valid ordered pair (1): e.g. "check the balance is sufficient" must come before "dispense the cash"; or "verify the PIN" before "allow withdrawal". Dependency explained (1): the later step relies on the result/output of the earlier step (you cannot safely dispense until the balance check has confirmed funds).

9. **[2]** Any two (1 each): easier to write/understand (smaller pieces); reusable — call the same routine in many places; easier to test/debug each part in isolation; can be shared among a team; easier to maintain/update one routine.

10. **[2]** Writing it once as a sub-procedure means it is defined in a single place (1), so it can be reused/called each round and any fix or change is made once and applies everywhere — avoiding duplicated, inconsistent code (1).

Section C (5 marks) 11. **[5]** Up to 5 from: Named sub-procedures, e.g. `readPupilData` → `calculateGrades` / `averageScores` → `generateComments` → `assembleReport` → `printReport` / `emailReport` (1 for sensible decomposition into named routines, 1 for a logical overall order). Identify a dependency, e.g. grades must be calculated before the report can be assembled, and data must be read before grades can be calculated (1). Explain the dependency: the later step uses the *output* of the earlier step (e.g. `assembleReport` needs the grades produced by `calculateGrades`), so the order is fixed (1). Recognise that some steps could be independent but the dependent ones cannot be reordered (1). Must apply to the reports scenario with named sub-procedures, not give a generic definition.

Total: 6 + 9 + 5 = 20